

第十一周

作业1

在分支限界算法中，根据从活结点表中选择下一扩展结点的不同方式可有几种常用分类，以下（ ）描述最为准确。

采用FIFO队列的队列式分支限界法

采用最小值堆的优先队列式分支限界法

采用最大值堆的优先队列式分支限界法

以上都常用，针对具体问题可以选择采用其中某种更为合适的方式

D

作业2

下面可以用分支限界法解决的问题有(多选): ()

- A. 旅行商问题
- B. 八数码问题
- C. 汉密尔顿回路
- D. 单源最短路径

ABCD

作业3

给定图 $G(V, E)$ ，以及其对应的一颗最小生成树，假定减少图中不属于最小生成树的一条边的权重，设计算法找出修改边后新图的最小生成树

算法思想：把修改的边加入到最小生成树里，这样在树中就会产生了一条含有修改边的回路。在这个回路中去掉权值最大的，就可以得到一颗新的最小生成树。

```

Dijkstra( $G, w, s$ )
1 InitializeSingleSource( $G, s$ )
2  $S \leftarrow \emptyset$ 
3  $Q \leftarrow V$ 
4 while  $Q \neq \emptyset$  do
5      $u \leftarrow \text{ExtractMin}(Q)$ 
6      $S \leftarrow S \cup \{u\}$ 
7     for each vertex  $v \in \text{Adj}[u]$  do
8         Relax( $u, v, w$ )

```

上图为Dijkstra算法的伪代码

将Dijkstra算法第四行改为：

while $|Q| > 1$

使得while循环执行 $|V|-1$ 次而不是 $|V|$ 次，算法是否仍然正确？

算法仍然正确。令 u 是队列 Q 中剩余的顶点。如果 u 不能从 s 可达，则有 $d[u] = \delta(s, u) = \infty$ 。如果 u 从 s 可达，则有一条路径 p ，它从 s 经过 x 再直接到 u ，由于 $d[x] = \delta(s, x)$ ，按照最短路径松弛性质，可得 $d[u] = \delta(s, u)$ 。故算法正确。

第十三周

作业1

试设计一个算法随机地产生范围在1~n中的m个随机整数，且要求这m个随机整数互不相同。

```
def randm(m, n):  
    a = []  
    for i in range(m):  
        num = random.randint(1, n)  
        while num in a:  
            num = random.randint(1, n)  
        a.append(num)  
    return a
```

作业2

试设计一个概率算法计算 $365! / 340! \cdot 365^{25}$ ，并精确到4位有效数字。

解答：首先化简原计算公式：

$$365! / (340! 365^{25}) = (341/365) * (342/365) * \dots * (364/365)$$

24 次随机试验为一次事件，

第 x 次试验随机在 $[1, 365]$ 中取值 r ，如果 $1 \leq r \leq (365 - x)$ 则该次试验为真，否则为假，

如果 24 次试验都为真，则该次事件为真，

进行 total 个事件，统计其中真事件的次数 count ，则计算结果为 $\text{total} / \text{count}$

```
#include <iostream>
#include <random>
using namespace std;
int randBirth() {
    static default_random_engine eng;
    static uniform_int_distribution<int> dis(1, 365);
    return dis(eng);
}
int main() {
    int t = 10;
    for (int i = 1; i <= t; ++i) {
        int total = i * 10000000;
        int count = total;
        for (int j = 0; j < total; ++j) {
            for (int k = 340; k <= 364; ++k) {
                if (randBirth() > k) {
                    count -= 1;
                    break;
                }
            }
        }
        cout << "total tests: " << total << ", result: " << double(count) / total
            << endl;
    }
    return 0;
}
```


作业3

设 p 是一个奇素数, $1 \leq x \leq p-1$, 如果存在一个整数 y , $1 \leq y \leq p-1$, 使得 $x \equiv y^2 \pmod{p}$, 则称 y 是 x 的模 p 平方根。例如, 63 是 55 的模 103 平方根。试设计一个求整数 x 的模 p 平方根的拉斯维加斯算法。

```
def LV(x,p):  
    y = random.randint(1,p-1)  
    flag = False  
    while(flag == False):  
        if x != (y * y) % p:  
            y = random.randint(1,p-1)  
        else:  
            flag = True  
    return y
```

作业4

设计一个时间复杂度为 $O(n^2)$ 的算法来产生 $1, 2, \dots, n$ 的一个随机排列。要求用文字描述算法的基本思路，并分析其时间复杂度。

每次随机从 1 到 n 中选择一个数，放入结果队列中；如果该数字已经在队列中，则重新随机选择直到该数数字不在队列中。由于每次随机选出一个数都有可能是之前选过的数，需要进行再次随机，因此对选出一个数的随机平均情况下需要随机 $O(n)$ 次，因此该方法的时间复杂度是 $O(n^2)$ 的

第十四周

作业1

解释什么是N问题，什么是NP问题

P问题：可以在多项式时间内解决的问题。

NP问题：可以在多项式的时间里验证一个解的问题。

作业2

证明如下问题是NP-完全的：

输入：两个图 $G1 = (V1, E1)$ 和 $G2 = (V2, E2)$ ：预算 b 。

输出：两个节点集合 $V1' \subseteq V1$ 和 $V2' \subseteq V2$ 和它们被移除后，将在两图中分别留下至少 b 个节点，且图的剩余部分完全一样

我们需要找到一个NP-完全问题规约到该问题上，从而证明出该问题是NP-完全的。我们选择团问题规约到该问题中。

考虑这个问题，当我们在图 $G1$ 中找到一个规模为 b 的团，便可以得到一个子图 $G2$ ，其中该子图 $G2$ 有 b 个顶点，且它是一个完全图。那么反过来说给出 $G1, G2$ ，如果我们能求解它们的最大公共子图，如果该子图的顶点数为 b ，便能得到一个规模为 b 的团，从而将团问题的实例 $(G1, b)$ 便能规约到在 $G1, G2$ (其中 $G2$ 是规模为 b 的完全图)求解最大公共子图实例 $(G1, G2, b)$ 的问题

证明：

设团问题的某个实例 $(G1, b)$ ，构造一个完全图 $G2$ ，顶点数为 b ，因此能通过团问题实例 $(G1, b)$ 映射到最大公共子图 $(G1, G2, b)$ 上。

①如果最大公共子图有一个解，则由于该公共子图实例要求顶点数 $\geq b$ ，且 $G2$ 的顶点数为 b ，故该公共子图的顶点数必为 b 。

由于 $G2$ 的顶点数为 b ，则该公共子图为 $G2$ ，且由于 $G2$ 为团，所以 $G1$ 所得的公共子图是规模为 b 的团，故可求得团问题的实例 $(G1, b)$ 是有解的。

②如果团问题的实例 $(G1, b)$ 有解，则 $G1$ 中存在一个规模为 b 的团 G' ，由于 $G2$ 也是规模为 b 的团，则 $G'=G2$ ，则可求得最大公共子图 $(G1, G2, b)$ 的解

因此，团问题 $(G1, b)$ 和最大公共子图 $(G1, G2, b)$ 的解释一致的，得证。

作业3

试写出完成下面计算的RAM和RASP程序：

- (1) 给定输入 n ，计算 $n!$ ；
- (2) 读入 n 个正整数（用0做结束标志），然后，按从大到小的顺序输出这 n 个数。

见 RAM RASP.pdf